



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

24th July 2026

**RealityCheck : An Evidence Grounded
Hallucination Correction Pipeline
for Large Language Models (LLMs)**

*A Practice School Report submitted to Manipal Academy of Higher Education
in partial fulfilment of the requirement for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Vishesh Goyal

225805242

Under the guidance of

Dr. Jayita Saha

Associate Professor

SOCE Dept. M.I.T, BENGALURU



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

Bengaluru

24th July 2026

CERTIFICATE

This is to certify that the project titled **RealityCheck : An Evidence Grounded Hallucination Correction Pipeline for Large Language Models** is a record of the bonafide work done by **Vishesh Goyal** (Reg. No. **225805242**) submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in **Computer Science and Engineering** of Manipal Institute of Technology Bengaluru, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Dr. Jayita Saha

Associate Professor

SOCE, M.I.T, BENGALURU

Dr. Dayananda P

DEAN,

SOCE M.I.T, BENGALURU

ACKNOWLEDGMENTS

I would like to take this opportunity to express my deepest gratitude to **Dr. Iven Jose**, Director of Manipal Institute of Technology, Bengaluru, for his constant encouragement and for fostering an environment that supports academic excellence and research-driven learning.

I am sincerely thankful to the Dean, **Dr. Dayananda P**, and the entire department of **School of Computer Engineering** for providing the necessary infrastructure, guidance and technical foundation that enabled me to undertake and complete this project successfully.

My heartfelt appreciation goes to my guide and mentor, **Dr. Jayita Saha** for her continuous support, insightful feedback and expert guidance throughout the course of this project. Her mentorship has been invaluable in shaping the direction and quality of my work. I also extend my thanks to the faculty members and staff who contributed to my learning experience, both directly and indirectly.

Finally, I would like to acknowledge **Manipal Institute of Technology, Bengaluru** for giving me this wonderful opportunity to work on a project that has greatly enriched my academic journey and professional growth.

ABSTRACT

Large Language Models (LLMs) are a remarkable step in today’s world of Artificial Intelligence with their exceptional capability at making contextually coherent, fluent responses across different domains. However, even after their fluent text generation, they still are susceptible to hallucinations or the generation of factually incorrect or unverifiable content with confidence. This can lead to severe consequences in the fields like legal practices, healthcare and academic research where misinformation can have severe avalanche effect. This problem has been tackled with different approaches like Retrieval Augmented Generation (RAG) and post-hoc detection frameworks. However, the problem remains that these systems address hallucination at generation phase or diagnose it after it has been delivered to user for the future correction of response. There exists no such widely used mechanism which provides inline correction of erred claims before the response is delivered to the user.

To solve this issue, we present RealityCheck. A modular, model agnostic, six phase hallucination correction pipeline. It functions as an additional verification layer instead of modifying the internal parameters. The system takes the LLMs’ responses, extracts factual claims, retrieves Wikipedia evidence through a cache first retrieval and verifies each claim through a multi signal framework using Natural Language Inference (NLI), semantic alignment and rule based reasoning. This produces a corrected response through deterministic answer synthesis. We evaluated the RealityCheck on TruthfulQA benchmark across IBM’s WatsonX Granite, Meta Llama and MistralAI Mistral models. RealityCheck achieved substantial improvements in claim level accuracy and hallucination recall across all the three models while giving low overcorrection rates. The results show the possibility of evidence backed, claim level correction pipeline for improving the factual reliability of LLM generated content in high stake deployment scenarios. Crucially, since RealityCheck operates without modifying the internal parameters of the underlying model, it is practically deployable alongside existing state-of-the-art LLMs requiring no architectural rework from developers, and extensible to additional evidence sources as industry requirements evolve.

Keywords: *Large Language Models, Hallucination Mitigation, Fact Verification, Retrieval-Augmented Verification, Natural Language Inference, Evidence-Grounded AI, Answer Synthesis, Trustworthy AI*

Table of Contents			
			Page No
Acknowledgement			II
Abstract			III
Chapter 1		INTRODUCTION	1
	1.1	Present-Day Scenario	1
	1.2	Motivation to Do the Project Work	2
	1.3	Objective of the Work	3
	1.4	Target Specifications	3
	1.5	Project Schedule	4
	1.6	Organization of the Project Report	5
Chapter 2		LITERATURE REVIEW	7
	2.1	Hallucination in Large Language Models	7
	2.2	Fact Verification and Knowledge-Grounded Systems	8
	2.3	Response Correction through External Knowledge Grounding	9
	2.4	Retrieval-Augmented and Wikipedia-Grounded Generation	9
	2.5	NLI-Based Verification Approaches	10
	2.6	Sentence Transformers for Semantic Decomposition	11
	2.7	Comparative Analysis and Research Gap	11
	2.8	Outcomes of the Literature Review	12
Chapter 3		METHODOLOGY	
	3.1	Introduction and Methodological Approach	14
	3.2	System Overview	15
	3.3	Development Progression and Design Evolution	15
	3.4	Dataset and Data Preparation	17
	3.5	LLM Response Generation	18

	3.6	Factual Claim Extraction and Semantic Decomposition	18
	3.7	Wikipedia-Grounded Evidence Retrieval	19
	3.8	Multi-Signal Claim Verification	19
	3.9	Evidence-Grounded Answer Synthesis	20
	3.10	Live Console and Pipeline Orchestration	21
	3.11	Experimental Evaluation Workflow	22
	3.12	Technology Used	22
	3.13	Evaluation Metrics	23
Chapter 4		RESULT ANALYSIS	24 - 29
Chapter 5		CONCLUSION AND FUTURE SCOPE	
	5.1	Brief Summary and Conclusion	30
	5.2	Future Scope	31
REFERENCES			32
PLAGIARISM AI REPORT			38
PROJECT DETAILS			40

LIST OF TABLES

Table No	Table Title	Page No
1	<i>Main results across all evaluated models. RC denotes the RealityCheck-corrected condition.</i>	16
2	<i>Claim-level verification verdict distribution across models.</i>	16
3	<i>Distribution of answer-synthesis outcomes across all evaluated responses.</i>	16
4	<i>Wikipedia evidence-retrieval performance.</i>	
5	<i>Incremental contribution of pipeline components to answer accuracy.</i>	
6	<i>Representative qualitative examples of RealityCheck corrections.</i>	

LIST OF FIGURES

Figure No	Figure Title	Page No
1	<i>Full architecture of the RealityCheck six-phase hallucination correction pipeline.</i>	3
2	<i>Claim-level verification flow in Phase 5 of the RealityCheck pipeline.</i>	4
3	<i>Answer synthesis logic in Phase 6 of the RealityCheck pipeline.</i>	11
4	<i>Comparison of raw LLM and RealityCheck-corrected performance across all evaluation metrics.</i>	17
5	<i>Distribution of claim-level verification verdicts across the three evaluated models.</i>	17

CHAPTER 1

INTRODUCTION

This chapter introduces the research area of factual reliability in Large Language Models (LLMs) and establishes the need for an evidence-grounded mechanism that can correct hallucinated content before it is delivered to a user. It first describes the present-day use of LLMs as general-purpose knowledge interfaces and explains why fluent but unsupported answers create a serious reliability problem. The chapter then identifies the limitations of existing hallucination-mitigation approaches, presents the motivation and objectives of the proposed work, defines the expected system specifications, outlines the major phases followed during the project, and concludes with the organization of the thesis.

The project lies at the intersection of Natural Language Processing, information retrieval, fact verification, Natural Language Inference (NLI), and trustworthy artificial intelligence. The proposed system, titled RealityCheck, is designed as an external verification and correction layer for LLM-generated responses. Instead of modifying or retraining the internal parameters of an LLM, it processes the generated answer after initial generation, decomposes it into factual claims, retrieves evidence from Wikipedia, verifies each claim using multiple complementary signals, and synthesizes a corrected response through deterministic logic. This design enables the framework to operate across different language models while retaining transparency, modularity, and reproducibility.

1.1 Present-Day Scenario

Large Language Models have rapidly become widely used interfaces for obtaining information, generating content, supporting education, assisting research, and interacting with domain-specific applications. Their success is largely driven by their ability to produce responses that are fluent, contextually coherent, and stylistically convincing. For many users, the quality and confidence of the generated language can make an LLM appear more authoritative than a conventional search system, even when the underlying answer has not been verified against a reliable source.

This apparent reliability creates a major technical concern, hallucination. In this context, hallucination refers to the generation of claims that are factually incorrect, contradicted by available evidence, or insufficiently verifiable, while being presented in a confident and linguistically polished manner. Because the false statement is often blended naturally with correct information, users may not recognize which part of the response requires verification. The risk is particularly serious in high-stakes domains such as healthcare, legal practice, education, academic research, and public communication, where a single fabricated citation, incorrect statistic, or unsupported factual claim can influence subsequent decisions and propagate further misinformation.

Several research directions have attempted to improve the factuality of generated responses. Retrieval-Augmented Generation (RAG) introduces external evidence during generation [1];

NLI-based approaches evaluate whether a claim is entailed, contradicted, or unsupported by a reference passage [2]; and knowledge-grounded systems attempt to constrain model output using external resources [3]. These approaches have produced meaningful improvements, but they do not completely address the practical requirement of correcting an already generated answer before it reaches the user.

Generation-time retrieval does not guarantee that the model will use the retrieved context correctly, and a response may still contain unsupported claims. Post-hoc hallucination detectors can identify unreliable content, but many operate primarily as evaluation tools and do not reconstruct a corrected answer. Self-verification techniques such as Chain-of-Verification remain dependent on the same model's parametric knowledge and require additional model calls [4]. Knowledge-graph correction systems can provide structured evidence but are limited by the availability and coverage of domain-specific graphs [6]. Wikipedia-grounded conversational systems move closer to practical verification, yet some approaches remove unverifiable content entirely, which may reduce the informational usefulness of the response [8].

The present-day requirement is therefore not merely to detect that a response may contain a hallucination, but to create an inline correction mechanism that verifies individual claims, preserves supported content, repairs contradicted content, and treats unverifiable content cautiously. Such a mechanism should remain independent of the internal architecture of the generating LLM, provide an auditable evidence trail, and avoid introducing new hallucinations during the correction stage.

1.2 Motivation to Do the Project Work

The principal motivation for this project is the gap between hallucination detection and practical response correction. Existing methods often improve factual grounding at the generation stage or identify errors after generation, but a user-facing system requires a dependable intermediate layer that intercepts the response, evaluates its factual claims, and delivers a corrected version before the answer is shown. RealityCheck is developed to address this requirement through a modular and model-agnostic pipeline.

- **Shortcomings in Previous Work and Reference Papers:** Existing approaches generally solve only a subset of the problem. RAG improves access to external context but cannot ensure that every generated claim follows the retrieved evidence. NLI-based verification can identify entailment or contradiction but may struggle with compound statements, multi-hop reasoning, temporal calculations, and poor evidence retrieval. Self-verification remains restricted by the originating model's own knowledge, while knowledge-graph methods are constrained by domain coverage. A unified system that combines claim extraction, external grounding, multi-signal verification, correction, and epistemic hedging is still required.
- **Importance of the Work in the Present Context:** LLMs are increasingly integrated into applications where users expect direct, trustworthy answers rather than a list of sources to inspect manually. In such settings, detecting false content after it has already been delivered is not sufficient. An inline evidence-grounded correction layer can reduce the

likelihood that confidently stated misinformation is accepted or propagated, thereby improving the practical safety of existing LLM deployments without requiring expensive retraining.

- **Uniqueness of the Methodology Adopted:** RealityCheck employs a six-phase pipeline in which each module has a clearly defined input and output. The framework performs response generation, factual claim extraction, cache-first evidence retrieval, claim-level verification, reasoning-based verdict refinement, and deterministic answer synthesis. Verification combines semantic similarity, NLI, rule-based support and contradiction overrides, and aggregation across multiple evidence chunks. The corrected answer is then constructed without invoking another LLM, reducing the possibility of a second hallucination during correction.
- **Significance of the Possible End Result:** The proposed system demonstrates how an external verification layer can improve the factual reliability of multiple LLMs without changing their internal weights or requiring model-specific fine-tuning. A successful implementation can serve as a deployable foundation for trustworthy question-answering systems and can be extended to additional models, evidence sources, and domain-specific verification modules.

1.3 Objective of the Work

- To design and implement RealityCheck as a modular, evidence-grounded, six-phase hallucination correction pipeline that operates as an external layer over existing Large Language Models.
- To decompose LLM-generated responses into discrete factual claims, identify whether each statement is checkable, and preserve contextual information required for accurate evidence retrieval.
- To develop a cache-first Wikipedia-grounded retrieval mechanism that obtains claim-specific evidence, reduces repeated external API calls, and supports reproducible experimental runs.
- To verify claims through a multi-signal framework combining semantic alignment, Natural Language Inference, rule-based support and contradiction handling, evidence aggregation, and deterministic temporal reasoning where required.
- To synthesize a corrected answer by retaining supported claims, repairing contradicted claims using retrieved evidence, and hedging or omitting claims for which sufficient evidence is unavailable, without using an additional LLM during the synthesis stage.
- To evaluate the proposed framework on the TruthfulQA benchmark [11] across IBM WatsonX Granite, Meta Llama, and MistralAI Mistral using claim accuracy, answer accuracy, hallucination recall, correction success rate, overcorrection rate, and evidence coverage.
- To assess whether the framework remains model-agnostic, auditable, extensible, and practically deployable without fine-tuning or modifying the internal parameters of the evaluated language models.

1.4 Target Specifications

The target specifications define the functional and non-functional requirements expected from

the RealityCheck framework. The system is intended to operate as a unified but modular pipeline in which every stage produces structured outputs that can be inspected independently. The final result must provide not only a corrected answer but also sufficient evidence and verdict information to explain why individual claims were retained, modified, softened, or excluded.

- **Model-Agnostic Operation:** The correction pipeline shall remain independent of the internal architecture of the generating model and shall support the evaluated IBM WatsonX Granite, Meta Llama, and MistralAI Mistral models without changing the downstream verification or synthesis logic.
- **Six-Phase Modular Architecture:** The implementation shall separate response generation, claim extraction, evidence retrieval, claim verification, verdict refinement, and answer synthesis into independently replaceable modules with clearly defined interfaces.
- **Claim-Level Processing:** The system shall decompose a response into factual units, distinguish checkable from non-checkable statements, and assign structured metadata needed for evidence retrieval and later analysis.
- **Evidence-Grounded Retrieval:** Relevant evidence shall be retrieved from Wikipedia through a cache-first strategy, local evidence storage, MediaWiki API access, and fallback search. Retrieved pages shall be segmented and ranked so that verification operates on claim-specific evidence rather than complete documents.
- **Multi-Signal Verification:** Each checkable claim shall be assessed using semantic similarity, NLI-based entailment or contradiction, rule-based overrides, and aggregated evidence signals. Temporal claims shall be handled through deterministic computation when appropriate.
- **Transparent Verdicts:** Each claim shall receive a structured verdict such as supported, contradicted, insufficient evidence, or non-checkable, together with the best available evidence and a human-readable reason for the decision.
- **Deterministic Answer Synthesis:** The final response shall preserve supported claims, replace contradicted claims with evidence-backed alternatives, and hedge or omit insufficiently verifiable claims. The correction stage shall not invoke an additional LLM, thereby limiting secondary hallucination and improving reproducibility.
- **Auditability and Reproducibility:** Intermediate outputs from every phase shall be persisted so that the complete decision path can be reviewed. Cached evidence and fixed experimental controls shall support repeatable evaluation.
- **Evaluation Requirements:** The system shall be assessed on TruthfulQA using both claim-level and answer-level metrics, with particular attention to hallucination recall, correction success, evidence coverage, and the avoidance of unnecessary modification of originally correct claims.
- **Extensibility:** The retrieval source, sentence-embedding model, NLI verifier, and LLM backend shall be replaceable without redesigning the complete pipeline, allowing future use with additional knowledge sources and application domains.

1.5 Project Schedule

The project work was organized into six sequential phases. The phases below describe the logical execution order; the exact calendar dates may be aligned with the academic semester

schedule in the final thesis.

- **Phase 1 - Problem Formulation and Literature Survey:** Study hallucination taxonomies, Retrieval-Augmented Generation, NLI-based fact verification, self-verification, knowledge-grounded correction, Wikipedia-grounded systems, and benchmarks for factual reliability.
- **Phase 2 - Dataset Preparation and System Design:** Prepare the TruthfulQA evaluation corpus, define the six-stage architecture, establish the structured data exchanged between modules, and select the language models, retrieval services, embedding models, and verification components.
- **Phase 3 - Core Module Development:** Implement LLM response generation, factual claim extraction, context enrichment, cache-first Wikipedia evidence retrieval, semantic chunk ranking, and persistent storage of intermediate outputs.
- **Phase 4 - Verification and Synthesis Integration:** Integrate semantic similarity, cross-encoder NLI, rule-based reasoning, evidence aggregation, temporal verification, and deterministic answer synthesis into an end-to-end correction pipeline.
- **Phase 5 - Experimental Evaluation and Analysis:** Execute controlled experiments across Granite, Llama, and Mistral; calculate the defined factuality and correction metrics; compare raw and corrected responses; and analyse retrieval failures, overcorrection, and other limitations.
- **Phase 6 - Documentation and Thesis Preparation:** Consolidate the architecture, implementation details, experimental findings, limitations, and future extensions into the final undergraduate thesis and supporting research article.

1.6 Organization of the Project Report

This thesis is organized into five chapters following the structure of the prescribed undergraduate project report format.

Chapter 1 (Introduction): Presents the current problem of hallucination in Large Language Models, the motivation for developing an inline correction mechanism, the objectives of RealityCheck, the target specifications, and the sequence of project activities.

Chapter 2 (Literature Review): Reviews hallucination taxonomies, Retrieval-Augmented Generation, NLI-based verification, Chain-of-Verification, knowledge-graph correction, Wikipedia-grounded systems, factuality benchmarks, and sentence-transformer methods. It concludes by identifying the research gap addressed by the proposed pipeline.

Chapter 3 (Methodology): Describes the TruthfulQA dataset, the complete six-phase RealityCheck architecture, response generation, factual claim extraction, evidence retrieval, claim verification, reasoning overrides, deterministic answer synthesis, Live Console orchestration, implementation technologies, and reproducibility controls.

Chapter 4 (Result Analysis): Presents the experimental setup, evaluation metrics, quantitative comparison of raw and RealityCheck-corrected responses across the three evaluated language models, claim-level verdict distributions, answer-synthesis outcomes, evidence-retrieval

performance, error analysis, limitations, and the significance of the findings.

Chapter 5 (Conclusion and Future Scope): Summarizes the outcomes of the work, evaluates the achievement of the stated objectives, discusses the practical implications of external evidence-grounded correction, and outlines future extensions to additional evidence sources, language models, domains, and deployment environment.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the principal research directions that form the theoretical and technical foundation of RealityCheck. The discussion begins with the nature of hallucination in Large Language Models (LLMs), followed by research on factual verification, knowledge-grounded generation, post-generation correction, retrieval-augmented systems, Natural Language Inference (NLI), and sentence-level semantic representation. The objective of this review is not only to describe previous methods, but also to identify how their individual strengths and limitations motivate the need for a unified, evidence-grounded, inline correction pipeline.

Research on factual reliability has progressed through several complementary paths. Some studies explain why hallucinations occur or attempt to detect uncertainty; others retrieve external information, verify individual claims, compare outputs from several models, or score the factual precision of generated text. However, these capabilities are commonly implemented as separate mechanisms. RealityCheck is positioned at the intersection of these areas by combining claim extraction, external evidence retrieval, multi-signal verification, contradiction handling, epistemic hedging, and deterministic answer synthesis in a model-agnostic framework.

2.1 Hallucination in Large Language Models

Hallucination has become a central reliability problem in natural language generation. Ji et al. [3] provide a detailed taxonomy that distinguishes intrinsic hallucinations, in which a generated statement contradicts the available source context, from extrinsic hallucinations, in which the model produces information that cannot be verified from the available evidence. This classification is important because it shows that hallucination is not a single error type. A response may directly oppose known information, fabricate an unsupported detail, or combine correct and incorrect statements in a manner that is difficult for the user to detect.

The same survey argues that hallucination is not merely an incidental malfunction of a particular model. It is closely related to the statistical objective of language models, which rewards linguistic plausibility and continuation quality rather than guaranteed factual fidelity [3]. An LLM can therefore produce a grammatically polished and contextually coherent answer even when the underlying factual content is false. This mismatch between fluency and truthfulness is especially dangerous because authoritative language can cause users to place greater trust in an incorrect statement than they would in a visibly uncertain or incomplete answer.

Farquhar et al. [2] approach hallucination detection through semantic entropy. Their method samples multiple answers to the same question, groups semantically equivalent responses, and estimates uncertainty over the resulting meaning clusters. High semantic entropy indicates that the model is producing inconsistent meanings and is therefore more likely to be uncertain or hallucinating. This work is significant because it moves beyond token-level confidence and provides a probability-based signal for identifying unreliable model outputs.

Although semantic entropy improves the theoretical and practical detection of uncertainty, it does not itself repair the unreliable statement. It also requires repeated sampling from the model,

which increases inference cost. The distinction between detecting that an answer may be unreliable and producing a verified replacement is therefore important. RealityCheck focuses on the latter requirement by obtaining external evidence and reconstructing the response at the claim level.

Dhuliawala et al. [4] proposed Chain-of-Verification (CoVe), a self-verification strategy in which the model first generates an initial answer, formulates verification questions for the claims in that answer, answers those questions independently to reduce self-reinforcing bias, and finally produces a revised response. CoVe demonstrated measurable reductions in hallucination across open-domain and list-based tasks, establishing that deliberate verification can improve factual quality even without changing model parameters.

The principal limitation of CoVe is that both the original response and the verification process remain bounded by the same model's parametric knowledge [4]. If the model lacks the required knowledge or has learned an incorrect association, self-questioning may reproduce the same error. The method also requires several additional LLM calls for each user query. In contrast, RealityCheck grounds verification in an external evidence source and performs deterministic answer synthesis without additional LLM calls during correction.

2.2 Fact Verification and Knowledge-Grounded Systems

Fact verification against structured or semi-structured knowledge sources has been a long-standing problem in Natural Language Processing. A verification system must identify the claim being evaluated, retrieve evidence that is both topically relevant and sufficiently specific, and determine whether that evidence supports, contradicts, or fails to resolve the claim. Errors at any of these stages can propagate to the final verdict, which makes claim decomposition and evidence selection as important as the verification classifier itself.

Thorne et al. [5] introduced FEVER, a large-scale benchmark containing 185,445 claims derived from Wikipedia. Each claim is labelled as Supported, Refuted, or NotEnoughInfo. This three-way scheme became an influential reference point for claim-level verification because it distinguishes between a claim that is contradicted and a claim for which adequate evidence is simply unavailable. Treating these outcomes separately prevents a system from incorrectly interpreting missing evidence as proof that a statement is false.

FEVER also helped establish Wikipedia as a practical grounding source for general-domain fact verification [5]. Wikipedia offers broad topical coverage, human-maintained articles, and retrievable passages that can be associated with individual claims. Nevertheless, a benchmark and its labels do not automatically provide a complete user-facing correction mechanism. A deployed system must additionally retrieve the appropriate pages, isolate claim-specific evidence, handle paraphrases and compound statements, and translate the verification outcome into a corrected and readable answer.

RealityCheck adopts the conceptual separation represented by FEVER while adapting it for inline response correction. Claims are classified as supported, contradicted, or insufficiently verifiable, while non-checkable content is handled separately. This distinction allows supported claims to be preserved, contradicted claims to be repaired, and unverifiable claims to be hedged

instead of being presented with unjustified certainty.

2.3 Response Correction through External Knowledge Grounding

A substantial portion of hallucination research concentrates on detection and evaluation, whereas active correction of generated content remains comparatively limited. Detecting a false claim is valuable, but a practical user-facing system must also determine what should replace that claim and how the replacement can be integrated into the original response without introducing a new error.

Guan et al. [6] proposed Knowledge Graph-based Retrofitting (KGR). The framework identifies factual statements in an LLM-generated response, validates them against a knowledge graph, and replaces incorrect statements with verified alternatives. KGR demonstrated that post-generation correction is viable and can be effective for compound reasoning tasks. It therefore provides important evidence that factual improvement need not be restricted to generation-time prompting or model retraining.

The practical reach of knowledge-graph correction is constrained by the completeness and quality of the graph used for verification [6]. Comprehensive graphs are not available for every domain, and a graph may omit contextual, emerging, or weakly structured information. RealityCheck addresses this coverage limitation by grounding verification in Wikipedia, a broad, continuously maintained, domain-agnostic free-text corpus. This choice trades the precision of a curated graph for substantially wider general-domain coverage and makes the correction pipeline applicable to a broader class of questions.

Min et al. [7] introduced FActScore for the fine-grained evaluation of long-form generated text. FActScore decomposes a response into atomic facts and evaluates each fact independently against a knowledge source. This approach is valuable because response-level accuracy can hide local errors; a long answer may appear broadly correct while containing several unsupported atomic claims. Atomic evaluation therefore provides a more precise view of factual reliability.

FActScore is primarily an evaluation metric rather than a pipeline-driven correction system [7]. It measures factual precision but does not alter the original answer or generate a verified replacement for an incorrect fact. RealityCheck builds upon the same claim-level perspective while extending it from measurement to intervention. The pipeline uses claim-level verdicts as direct instructions for retaining, correcting, omitting, or hedging the corresponding parts of the final response.

2.4 Retrieval-Augmented and Wikipedia-Grounded Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [1], is one of the most prominent methods for grounding language model outputs in external knowledge. RAG retrieves relevant passages and conditions the generation process on those passages, thereby providing the model with information beyond its internal parameters. The approach has demonstrated improved performance on knowledge-intensive tasks and established retrieval as a major component of modern factuality-oriented language systems.

However, RAG operates primarily at the generation stage [1]. Retrieved evidence informs what

the model may generate, but it does not guarantee that the model will use the evidence correctly, resolve contradictions between passages, or avoid introducing an unsupported statement. Furthermore, a conventional RAG pipeline is not designed to retrospectively analyse and correct an erroneous claim that is already present in a completed response. Retrieval alone therefore improves access to knowledge but does not constitute a complete correction mechanism.

WikiChat [8] moves closer to the objective of evidence-grounded conversational reliability. Its pipeline extracts claims from LLM-generated responses, verifies them against live Wikipedia content, and filters unverified facts before composing the final answer. This work demonstrates the practical value of claim-level Wikipedia grounding and shows that external verification can be integrated into a conversational system rather than being used only as an offline metric.

Despite this alignment, WikiChat is primarily implemented as a conversational agent based on few-shot prompting of a single underlying model [8]. It does not provide the same modular, model-agnostic correction pipeline across multiple LLM backends. In addition, removing unverifiable claims entirely can lead to information loss, particularly when a cautious statement would still be useful to the user. RealityCheck instead introduces epistemic hedging, allowing insufficiently verifiable content to be softened rather than automatically asserted or discarded.

Dey et al. [9] proposed Uncertainty-Aware Fusion (UAF), an ensemble strategy that combines responses from multiple LLMs according to estimates of confidence and factual accuracy. The approach benefits from diversity across model knowledge and can improve an answer when one model compensates for the weakness of another. Ensemble agreement is therefore a useful signal for reducing dependence on a single model.

Nevertheless, UAF remains limited to the collective parametric knowledge of the participating models and does not independently verify their claims against an external ground-truth source [9]. Asking the same question to multiple LLMs also increases token usage and operational cost. RealityCheck supplements model output with evidence that is independent of model confidence and avoids additional LLM calls during the verification and synthesis stages.

2.5 NLI-Based Verification Approaches

Natural Language Inference models classify the logical relationship between a premise and a hypothesis as entailment, contradiction, or neutral, and are widely used in hallucination detection and factual consistency analysis [2]. In a claim-verification pipeline, the retrieved evidence passage is treated as the premise and the generated factual claim is treated as the hypothesis. Entailment indicates support, contradiction indicates conflict, and a neutral result indicates that the evidence does not resolve the claim.

The usefulness of NLI is strongly dependent on the quality of claim decomposition and evidence retrieval. An accurate classifier cannot produce a reliable verdict when the evidence is irrelevant, incomplete, or expressed at an unsuitable level of granularity. NLI models also struggle with compound claims and multi-hop reasoning, where the truth of one statement depends on several facts drawn from different sources [3]. A single evidence chunk may support only one part of the claim, causing the overall relation to be classified as neutral or incorrectly contradicted.

Further difficulties arise with paraphrased statements, negation, numerical or temporal reasoning,

and claims containing multiple entities. These failure modes show that NLI should not be treated as an isolated factuality oracle. The proposed work therefore uses NLI as one signal within a broader verification framework, combining it with semantic relevance, rule-based support and contradiction checks, multi-evidence aggregation, and deterministic handling for specific temporal claims.

Semantic-similarity-based decomposition is also used to reduce the effect of compound responses on verification. By dividing an answer into smaller factual units and retrieving evidence for each unit independently, the system increases the likelihood that the premise supplied to the NLI model corresponds directly to the claim being evaluated. This design responds to the limitations identified in existing NLI-based approaches without discarding their structured entailment framework.

2.6 Sentence Transformers for Semantic Decomposition

Sentence-level semantic representations have become a standard component of NLP systems that require fine-grained comparison, retrieval, clustering, or decomposition of text. Unlike lexical matching, embedding-based methods can identify semantically related statements even when they use different words or syntactic structures. This capability is particularly relevant to factual verification because an LLM claim and its supporting evidence are rarely expressed in exactly the same form.

Reimers and Gurevych [10] introduced Sentence-BERT (SBERT), a modification of BERT that uses siamese and triplet network structures to generate semantically meaningful sentence embeddings. The resulting vectors can be compared efficiently using cosine similarity or related vector operations. SBERT substantially reduces the computational cost of sentence-pair comparison relative to repeatedly applying a full cross-encoder to every possible pair.

The multi-qa-MiniLM-L6-dot-v1 model from the sentence-transformers family is a lightweight architecture designed for semantic retrieval tasks [10]. In the present work, sentence-level embeddings support the decomposition of LLM responses into discrete factual units and the ranking of retrieved evidence chunks against claim-specific queries. This enables downstream verification to operate on focused evidence rather than entire documents.

Sentence embeddings provide semantic alignment rather than factual proof. A claim and an evidence passage may be topically similar while still contradicting one another. Consequently, embedding similarity is used as a relevance and decomposition mechanism, not as the final verdict. It narrows the evidence space and improves claim-evidence matching before NLI and rule-based reasoning determine whether the evidence supports or contradicts the claim.

2.7 Comparative Analysis and Research Gap

The reviewed literature represents substantial progress across complementary dimensions of the hallucination problem. Ji et al. [3] establish a theoretical taxonomy and explain the systemic nature of hallucination, while Farquhar et al. [2] provide a probabilistic uncertainty signal through semantic entropy. RAG [1] demonstrates the benefit of external retrieval at generation

time, and CoVe [4] shows that structured self-verification can improve an initial model answer. FEVER [5] formalizes claim-level verification labels, KGR [6] demonstrates external knowledge-based correction, FActScore [7] enables atomic factuality evaluation, WikiChat [8] integrates Wikipedia-grounded verification into dialogue, UAF [9] exploits model diversity, and Sentence-BERT [10] provides an efficient semantic representation mechanism.

Despite these advances, no single reviewed approach simultaneously performs detection, independent external grounding, inline correction, and epistemic handling of unverifiable claims within a modular, model-agnostic pipeline. RAG intervenes before or during generation but cannot guarantee correction of an error already present in the answer [1]. CoVe remains constrained by the originating model's parametric knowledge and requires repeated LLM calls [4]. Knowledge graph approaches provide structured correction but depend on curated domain resources that are not universally available [6]. FActScore measures factuality without modifying the output [7], while WikiChat may remove unverifiable claims and thereby sacrifice informational completeness [8]. Ensemble fusion improves consensus but does not provide evidence independent of the participating models [9].

The research gap is therefore the absence of a practical verification layer that can be placed between an existing LLM and the end user, analyse a completed answer at the claim level, retrieve external evidence, classify each claim conservatively, and reconstruct a corrected response without introducing a second generative failure. Such a framework should remain independent of the generating model, preserve an auditable decision trail, minimise additional model calls, and allow its retrieval and verification modules to be replaced as deployment requirements evolve.

RealityCheck addresses this gap by grounding verification in a broad, domain-agnostic Wikipedia corpus, correcting contradicted claims inline, preserving supported claims, and representing insufficiently verifiable claims through epistemic hedging. Its modular architecture also permits the LLM backend, evidence source, embedding model, and NLI verifier to be changed without redesigning the entire pipeline. The framework thus combines capabilities that earlier works generally treat as isolated tasks.

2.8 Outcomes of the Literature Review

The literature review leads to the conclusion that hallucination mitigation cannot be reliably solved through a single technique. Retrieval improves access to external information but does not guarantee faithful use of that information. NLI provides a structured verification signal but is sensitive to evidence quality and complex reasoning. Self-verification reduces some errors but remains limited by model knowledge and repeated inference cost. Knowledge graphs support precise correction in well-covered domains, while Wikipedia offers broader but less structured coverage. Atomic factuality metrics reveal local errors but do not automatically repair them.

These findings justify the design of RealityCheck as a six-phase pipeline in which retrieval, semantic decomposition, NLI, rule-based reasoning, evidence aggregation, and deterministic synthesis operate as complementary stages. The reviewed work also supports three central design decisions of the project: verification must be grounded in evidence external to the generating model; correction must occur before the response is delivered to the user; and unverifiable

claims must be treated conservatively rather than converted into confident assertions.

The following chapter presents the methodology and architecture through which these design requirements are implemented in RealityCheck, including the dataset, response generation layer, claim extraction process, cache-first Wikipedia retrieval, multi-signal claim verification, and deterministic answer synthesis.

CHAPTER 3

METHODOLOGY

This chapter describes the research methodology, system architecture, experimental workflow, implementation decisions, and development progression of RealityCheck. The proposed system is a modular, evidence-grounded, six-phase pipeline designed to intercept factual errors in Large Language Model (LLM) responses before those responses are delivered to the user. The methodology combines controlled response generation, factual claim extraction, Wikipedia-grounded evidence retrieval, multi-signal claim verification, deterministic verdict assignment, and evidence-grounded answer synthesis. The chapter also explains the use of the TruthfulQA benchmark, the evaluated language models, the Live Console, caching and rate-control mechanisms, and the practical decisions recorded during project development.

The methodology was deliberately developed as an external verification layer rather than as a modification to the internal parameters of an LLM. Consequently, the framework does not require fine-tuning, retraining, or architectural access to the selected model. This design supports model independence, reproducibility, modular replacement of individual components, and practical deployment alongside existing LLM services.

3.1 Introduction and Methodological Approach

RealityCheck follows a pipeline-driven experimental methodology. A user question is first submitted to a selected LLM, after which the generated response is decomposed into factual claims. Evidence is retrieved independently from Wikipedia, and each claim is assessed using semantic relevance, Natural Language Inference (NLI), rule-based reasoning, and aggregated support or contradiction signals. The final response is then reconstructed so that supported claims are preserved, contradicted claims are corrected, and claims for which sufficient evidence cannot be obtained are either hedged or omitted.

The central methodological principle is separation of generation from verification. The LLM produces the initial response, but it is not treated as the authority for validating its own output. Verification is instead grounded in an external evidence source. This separation reduces self-reinforcing errors and provides an auditable chain from the original answer to the retrieved evidence, claim verdicts, and final corrected response.

The research was organised around the following methodological requirements:

- The system must be model-agnostic and must process outputs from IBM WatsonX Granite, Meta Llama, and MistralAI Mistral through the same downstream logic.
- Verification must operate at the claim level rather than assigning a single coarse label to an entire answer.
- Evidence must be retrieved from an external and broadly available knowledge source using a reproducible cache-first strategy.
- Contradiction detection must not depend exclusively on one classifier; multiple complementary signals must be fused.
- The correction stage must avoid generating a second hallucination while rewriting the answer.
- All intermediate artefacts must be stored so that a complete decision trail can be inspected after an experimental run.

3.2 System Overview

RealityCheck is structured as six sequential phases with clearly defined inputs and outputs. The first phase generates the initial LLM response. The second phase extracts and normalises factual claims. The third phase retrieves relevant Wikipedia evidence. The fourth phase verifies each claim against the retrieved evidence. The fifth phase applies reasoning overrides and assigns a final verdict. The sixth phase synthesises the corrected answer. A Live Console provides a unified interface through which these phases can be executed as one coherent system.

Each phase is independently replaceable. For example, Wikipedia may later be replaced or supplemented by a domain-specific corpus; the selected NLI cross-encoder may be changed without modifying claim extraction; and additional LLM backends may be connected without altering the verification and answer-synthesis stages. This modularity is important because hallucination mitigation is not a single-model problem. It requires coordinated handling of generation, retrieval, reasoning, uncertainty, and response construction.

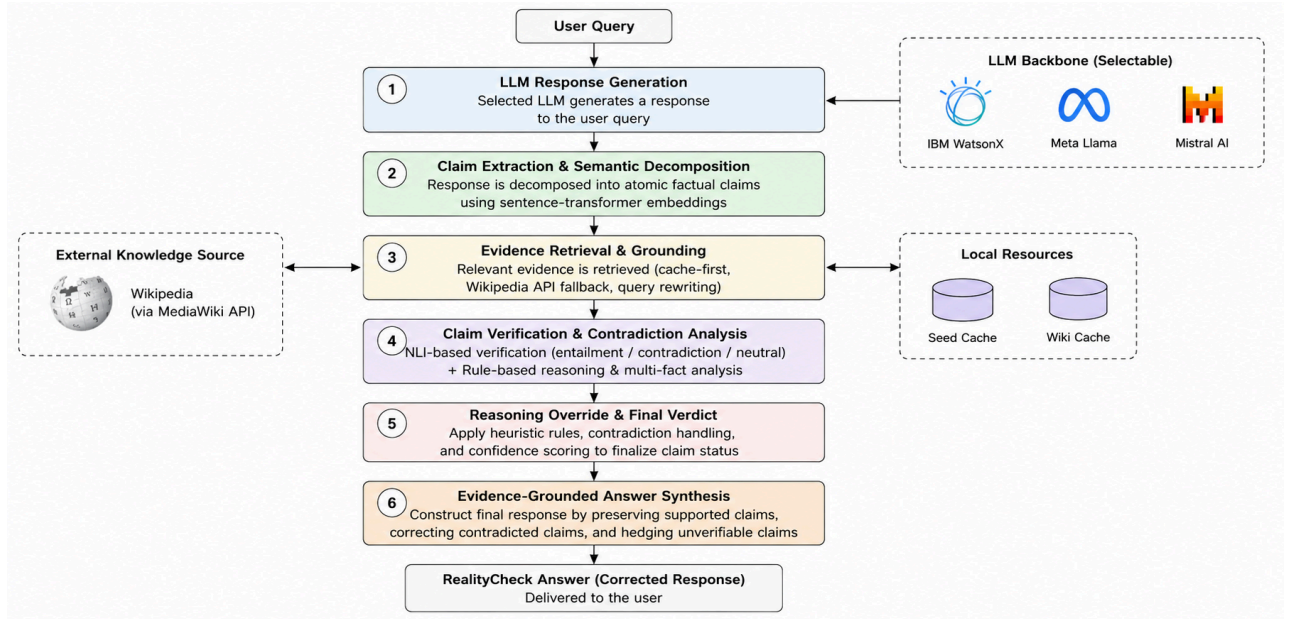


Figure 1 : Full architecture of the RealityCheck six-phase hallucination correction pipeline.

Figure 1 presents the complete architecture of the RealityCheck six-phase hallucination correction pipeline. It shows the flow from the user query to selectable LLM backends, claim extraction, evidence retrieval, claim verification, reasoning overrides, and deterministic answer synthesis. It also illustrates the interaction with the external Wikipedia source and the local seed and wiki caches.

3.3 Development Progression and Design Evolution

The project was implemented incrementally. The development log records a sequence of design-locking, module construction, orchestration, and evaluation activities. Early decisions were revised where necessary as the experimental system matured; therefore, the final journal configuration represents the consolidated implementation rather than every preliminary model choice considered during development.

Phase 1: Benchmark, Model, and Retrieval-Source Locking

The first development phase established the experimental foundation. TruthfulQA was selected as the benchmark because it contains questions designed to induce plausible but factually incorrect answers. Wikipedia was selected as the initial evidence source because it is broad, continuously maintained, accessible through public APIs, and suitable for claim-level retrieval. The first pipeline specification was: obtain a TruthfulQA question, request an answer from an LLM, extract factual claims, retrieve Wikipedia evidence, compare claims with evidence, flag hallucinated claims, rewrite them, and compare the factual quality of the original and corrected responses.

At this stage, the decision engine was specified through three possible claim labels. Evidence below a minimum relevance threshold was labelled insufficient. Evidence with a contradiction score exceeding the contradiction threshold led to a contradicted verdict. Evidence with a support score exceeding the support threshold led to a supported verdict. All remaining cases were conservatively treated as insufficient rather than being forced into an unsupported binary decision.

Phase 2: Data Preparation and Experiment Skeleton

The second phase created the experiment runner and established repeatable model querying. A fixed random seed and configurable sample size were used so that the same subset of TruthfulQA questions could be submitted across model runs. Preliminary experiments used a sample of twenty questions and generated structured output records containing the question, raw response, cleaned response, and execution metadata.

Representative commands used during development were:

```
PYTHONPATH=src python src/experiment_runner.py --model llama --sample-size 20 --seed 15
PYTHONPATH=src python src/experiment_runner.py --model mistral --sample-size 20 --seed 15
PYTHONPATH=src python src/experiment_runner.py --model granite --sample-size 20 --seed 15
```

Phase 3: Claim Extraction Module

The third development phase converted complete model answers into structured claim records. The module accepted the JSON output from Phase 2, segmented each response, identified checkable factual content, and produced model-specific claim files. This phase was essential because evidence retrieval and NLI verification become unreliable when applied to an entire answer containing several independent assertions.

```
PYTHONPATH=src python src/phase3_runner.py --input
../Phase_2/outputs/llama_20_chatfixed.json --output outputs/model_claims_llama.json

PYTHONPATH=src python src/phase3_runner.py --input
../Phase_2/outputs/mistral_20_chatfixed.json --output
outputs/model_claims_mistral.json

PYTHONPATH=src python src/phase3_runner.py --input
../Phase_2/outputs/granite_20_chatfixed.json --output
outputs/model_claims_granite.json
```

Phase 4: Evidence Retrieval and Grounding

The fourth phase moved the system from hallucination identification toward correction by attaching evidence to every checkable claim. Operational controls were introduced to comply with Wikipedia API

usage constraints and to improve reliability. These controls included a descriptive user-agent, a delay between requests, a maximum number of queries per claim, and a bounded retry policy for HTTP 429 rate-limit responses. Candidate pages were retrieved, ranked, reduced to the most relevant pages, chunked, and scored so that only the strongest passages were forwarded to verification.

```
export REALITYCHECK_USER_AGENT="RealityCheck/0.7 (student research project; contact: your_email@example.com) "
```

```
export REALITYCHECK_WIKI_DELAY=2.5
```

```
export REALITYCHECK_MAX_QUERIES_PER_CLAIM=3
```

```
export REALITYCHECK_MAX_429_PER_CLAIM=2
```

```
PYTHONPATH=src python src/phase4_runner.py --inputs <claim-files> --output-dir outputs  
--top-pages 3 --candidate-pages 8 --top-chunks 5
```

Phase 5: Verification, Integration, and Stress Testing

The fifth development phase integrated retrieved evidence with the claim-verification engine. The system was stress-tested as a coherent pipeline rather than as isolated scripts. Configurable thresholds were exposed for support, contradiction, minimum evidence relevance, and maximum evidence chunks. During development, representative values included a support threshold of 0.65, a contradiction threshold of 0.65, a minimum evidence score of 0.52, and a limit of five chunks. These values were treated as experimental parameters and were evaluated together with rule-based overrides rather than as universal constants.

```
PYTHONPATH=src python src/phase5_runner.py --inputs <evidence-enriched-claim-files>  
--output-dir outputs  
--support-threshold 0.65 --contradiction-threshold 0.65 --min-evidence-score 0.52  
--max-chunks 5
```

Phase 6: Deterministic Synthesis and Final Output

The sixth phase consumed the verified claim records and produced corrected answers. Although the earliest project plan considered asking an LLM to rewrite the answer using corrected evidence, the final methodology adopted deterministic answer synthesis. This was an important design improvement: using another generative call during correction could introduce new unsupported content. Deterministic templates and evidence-derived substitutions instead preserve reproducibility and prevent secondary hallucination during the final stage.

```
PYTHONPATH=src python src/phase6_runner.py --inputs <verified-claim-files>  
--output-dir outputs
```

3.4 Dataset and Data Preparation

The TruthfulQA benchmark [11] is used as the evaluation corpus. It contains 817 questions across 38 categories, including health, law, finance, history, popular misconceptions, and conspiracy theories. The benchmark is specifically designed to test whether a model reproduces widely held false beliefs rather

than merely failing due to lack of factual knowledge. This makes it suitable for evaluating a pipeline whose purpose is to detect and correct fluent, confident, but factually incorrect responses.

The raw dataset is preprocessed before model querying. Whitespace is normalised, structured answer fields are parsed, and the generation and multiple-choice validation information is merged into a unified record format. Stable question identifiers are preserved so that outputs from response generation, claim extraction, retrieval, verification, and answer synthesis can be joined reliably during evaluation.

A stratified random sample is drawn from the preprocessed corpus using a fixed random seed. Fixing the seed ensures that every evaluated model receives the same questions, supporting a controlled comparison. The initial development log demonstrates this procedure using a sample size of twenty questions and seed 15; the same reproducibility principle is retained for larger experimental runs.

3.5 LLM Response Generation

Each selected TruthfulQA question is submitted to the chosen LLM through a structured two-part prompt. The system prompt instructs the model to act as a factual question-answering assistant and constrains the response to a maximum of three sentences. It prohibits question restatement, unnecessary meta-commentary, and unsolicited disclaimers, while directing the model to acknowledge uncertainty rather than fabricate an answer for subjective or time-sensitive questions. The user prompt requests a direct response without excessive preamble or formatting.

The generated response is cleaned to remove markdown symbols and other formatting noise. A response-quality check flags outputs that are empty, abnormally short, or evasive. The raw API output, cleaned text, model identifier, question identifier, and quality flags are persisted. These records provide the input to claim extraction and permit direct comparison between the uncorrected model output and the final RealityCheck response.

The final experimental configuration evaluates IBM WatsonX Granite, Meta Llama, and MistralAI Mistral under identical prompting and generation constraints. No model-specific fine-tuning, retraining, or customised prompt optimisation is performed. This control is necessary to ensure that differences in outcomes reflect model behaviour and pipeline performance rather than unequal experimental treatment.

3.6 Factual Claim Extraction and Semantic Decomposition

The cleaned LLM response is decomposed into discrete factual units. Sentence segmentation is performed using punctuation-aware splitting. Each sentence is then evaluated for checkability, defined as whether the sentence expresses a factual assertion that can reasonably be verified against external evidence. Subjective statements, opinions, explicit uncertainty, and statements without a stable factual interpretation are marked non-checkable and excluded from ordinary evidence verification.

A sentence is treated as potentially checkable when it contains linguistically motivated factual cues, such as a copular or auxiliary verb, a named entity, a four-digit year, or a domain-relevant factual keyword. The module also detects context-dependent fragments, including short sentences beginning with referential pronouns or connective adverbs. For such claims, the original question and preceding sentence are incorporated into the evidence query to reduce ambiguity and resolve anaphoric references.

Each extracted claim is normalised and labelled as essential or supporting based on its position, lexical importance, and relation to the central answer. The output record contains the original sentence, normalised claim, checkability status, importance label, contextual information, and enriched retrieval

query. Sentence-level embeddings generated using the multi-qa-MiniLM-L6-dot-v1 sentence-transformer model [10] support semantic decomposition and downstream evidence matching.

3.7 Wikipedia-Grounded Evidence Retrieval

Evidence retrieval follows a cache-first, three-layer strategy. First, the system consults a local seed evidence bank containing curated Wikipedia content for frequently occurring TruthfulQA topics. When the seed bank supplies sufficient evidence, external requests are avoided. Second, if local evidence is inadequate, the Wikipedia MediaWiki Action API is queried using the enriched claim query. An OpenSearch fallback is triggered when the primary request fails to return useful pages. Third, when a TruthfulQA record contains a direct Wikipedia source URL, that page is treated as a high-priority source.

The retrieval configuration may obtain up to eight candidate pages for a claim and retain the three most relevant pages for detailed processing. Retrieved page text is divided into fixed-size overlapping chunks. Each chunk and claim query are embedded into a shared semantic space, and similarity scores are computed. The highest-ranked chunks are retained as the claim-specific evidence set. The journal configuration reports top-ten chunks during the formal experimental setup, while the development log records top-five settings during early module testing; this distinction reflects parameter refinement between development and final evaluation.

All retrieved pages and API responses are cached on disk. Caching reduces repeated network calls, improves latency, limits exposure to rate throttling, and enables reproducible reruns using the same evidence. A descriptive API user-agent, request delay, query limits, and bounded retry logic are used to ensure responsible interaction with Wikipedia services.

3.8 Multi-Signal Claim Verification

Every checkable claim is verified against its evidence set through a fusion of complementary signals. A single NLI score is insufficient because retrieval errors, paraphrases, negation, compound statements, and multi-hop reasoning can cause unreliable classification. RealityCheck therefore combines semantic relevance, NLI classification, high-precision rules, and evidence aggregation.

The first signal is semantic similarity between the claim and evidence chunk. It acts as a relevance gate so that unrelated text cannot produce a misleading support or contradiction score. The second signal is generated by a cross-encoder NLI model, cross-encoder/nli-deberta-v3-base, which classifies each evidence-chunk and claim pair as entailment, contradiction, or neutral. The third signal consists of rule-based support and contradiction overrides targeting known failure modes such as misattributed quotations, historical misconceptions, negated claims, and compound multi-entity assertions. The fourth signal aggregates evidence across multiple chunks using a Noisy-OR formulation, allowing several medium-strength passages to provide joint support.

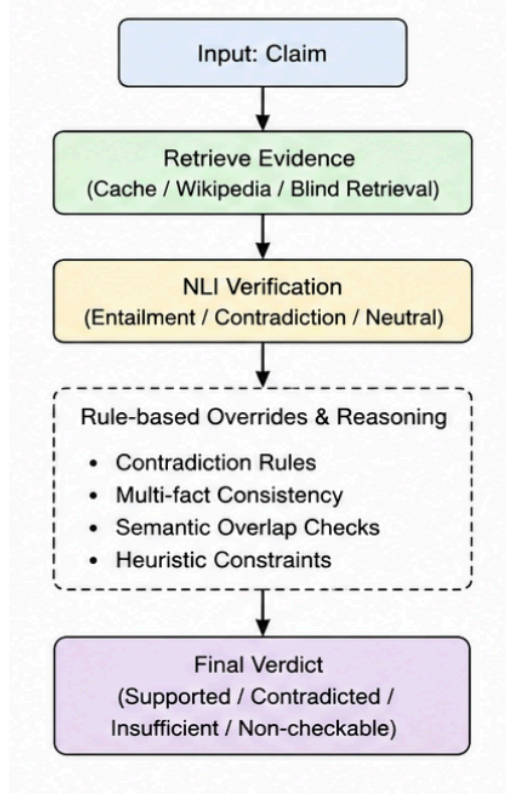


Figure 2 : Claim-level verification flow in Phase 5 of the RealityCheck pipeline.

Figure 2 shows the claim-level verification flow: claim input, evidence retrieval, NLI verification, rule-based overrides and reasoning, and assignment of the final verdict. The output label is one of supported, contradicted, insufficient evidence, or skipped/non-checkable. The record also stores the strongest supporting and contradicting passages together with a structured explanation of the verdict.

Temporal claims involving age or date computation are handled through a separate deterministic sub-module. Instead of asking an NLI model to infer arithmetic, the system extracts date evidence and performs the required computation directly. This avoids a known weakness of textual entailment models on numerical and temporal reasoning.

Decision Logic

The high-level decision logic used during implementation is conservative. A claim is not marked supported or contradicted unless evidence first satisfies the minimum relevance requirement. In conceptual form, the decision process is:

```

IF relevance < threshold_r: label = INSUFFICIENT
ELSE IF contradiction_score > threshold_c: label = CONTRADICTED
ELSE IF support_score > threshold_s: label = SUPPORTED
ELSE: label = INSUFFICIENT

```

The practical implementation supplements this threshold logic with rule-based overrides, aggregation across chunks, and explicit handling of non-checkable claims. The conservative fallback to insufficient evidence is intentional. It is safer to acknowledge uncertainty than to force an unsupported claim into a correct or incorrect category.

3.9 Evidence-Grounded Answer Synthesis

The final corrected response is constructed by a deterministic answer-synthesis module that does not invoke an LLM. Supported claims are retained without modification. Contradicted claims are replaced using evidence-grounded correction templates and the strongest contradicting passage. Claims with insufficient evidence are removed or rewritten using epistemic qualifiers so that potentially useful information is not presented as established fact. Non-checkable content is preserved when it does not introduce a factual risk.

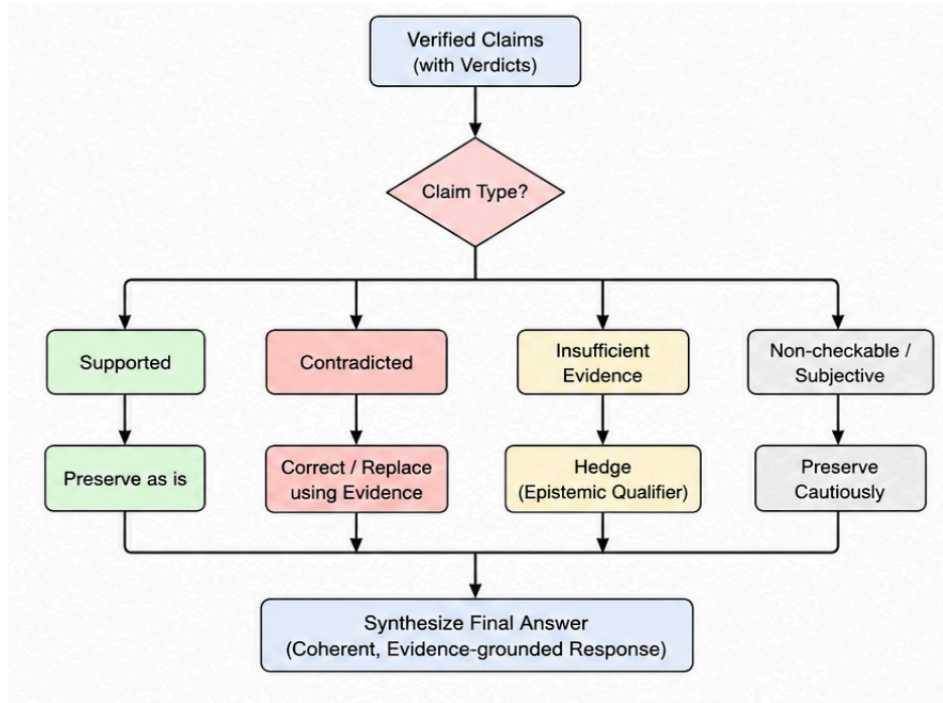


Figure 3 : Answer synthesis logic in Phase 6 of the RealityCheck pipeline.

Figure 3 illustrates the answer-synthesis logic. After claim-level actions are applied, corrected sentences are deduplicated and joined into a coherent response. The final answer receives one of six human-readable status labels: fully supported, corrected from contradiction, partially correct repaired, partially supported with omissions, insufficient evidence, or not checkable. This label summarises the degree to which the initial answer required intervention.

Deterministic synthesis was selected over LLM-based rewriting because the latter could reintroduce hallucinations after verification. By using explicit templates and retrieved evidence, the correction stage remains auditable and reproducible. This design decision is one of the major differences between the preliminary project plan and the final implemented methodology.

3.10 Live Console and Pipeline Orchestration

The six phases are integrated through a Live Console that permits end-to-end execution from a single user query. The console accepts free-text input, allows selection of a supported LLM, executes all phases sequentially, and displays or stores the corrected response. The development environment can be activated and the Granite-backed console launched using:

```
source venv/bin/activate
python Live_Console/src/realitycheck_console.py --model granite
```

Every phase writes structured intermediate output to a timestamped run directory. This design creates an audit trail containing the raw response, extracted claims, retrieval queries, evidence passages, NLI

outputs, rule-based overrides, final verdicts, and synthesised answer. The Live Console therefore serves both as a demonstration interface and as a practical deployment shell for the complete pipeline.

3.11 Experimental Evaluation Workflow

Evaluation is performed by running the same randomly selected TruthfulQA questions through the raw LLM condition and the complete RealityCheck condition. Phase 5 verification records, Phase 6 corrected-answer records, and the processed TruthfulQA ground truth are supplied to the evaluation script. The output directory contains aggregate metrics and model-wise result files.

```
python Evaluation/src/evaluate_realitycheck.py \
  --phase6 <phase6-corrected-output-files> \
  --phase5 <phase5-verification-output-files> \
  --truthfulqa Phase_2/data/truthfulqa_processed.json \
  --output-dir Evaluation/outputs
```

The formal evaluation uses claim-level accuracy, answer-level accuracy, hallucination detection recall, correction success rate, overcorrection rate, and evidence coverage. The same prompt structure and question set are used for all models. RealityCheck remains external to the underlying models throughout evaluation; no internal weights are modified. Therefore, improvement between the raw and corrected conditions can be attributed to the evidence-grounded verification and deterministic synthesis pipeline within the limits of the selected benchmark and experimental configuration.

3.12 Technology Used

The system is implemented in Python and organised as independent executable phases. IBM WatsonX provides hosted access to the evaluated Granite, Llama, and Mistral models. Sentence-level representation and semantic retrieval are implemented through the sentence-transformers library using multi-qa-MiniLM-L6-dot-v1 [10]. Claim entailment and contradiction analysis use the cross-encoder/nli-deberta-v3-base NLI model. Wikipedia evidence is accessed through the MediaWiki Action API and OpenSearch API. JSON files are used to persist intermediate outputs, and local disk caches store seed evidence, retrieved pages, and repeated query results.

The principal technology components and their roles are summarised below:

Technology / Component	Purpose
Python	Core implementation language and phase orchestration.
IBM WatsonX	Hosted inference environment for Granite, Llama, and Mistral.
TruthfulQA	Benchmark corpus for inducing and evaluating truthful versus misleading responses [11].
Wikipedia MediaWiki and OpenSearch APIs	External evidence retrieval and fallback search.
Sentence Transformers	Claim and evidence embeddings for semantic matching [10].
Cross-Encoder NLI	Entailment, contradiction, and neutral classification.
Local seed and wiki caches	Latency reduction, API-load control, and reproducibility.

Live Console	Interactive end-to-end model selection and pipeline execution.
JSON-based intermediate records	Auditability and reproducible phase-to-phase data transfer.

3.13 Evaluation Metrics

System performance was measured at both claim level and answer level. Six metrics were used so that improvements in factual accuracy could be interpreted alongside hallucination sensitivity, correction quality, retrieval completeness, and the risk of modifying already-correct content.

Claim-Level Accuracy: the proportion of individual factual claims for which the pipeline produces the correct verification outcome relative to the ground-truth label.

$$\text{Claim Accuracy} = \frac{\text{Correctly Verified Claims}}{\text{Total Claims}}$$

Answer-Level Accuracy: the proportion of final synthesised responses judged factually correct as complete answers.

$$\text{Answer Accuracy} = \frac{\text{Correct Final Responses}}{\text{Total Responses}}$$

Hallucination Detection Recall: the proportion of hallucinated claims correctly identified by the verification pipeline.

$$\text{Hallucination Recall} = \frac{\text{Detected Hallucinated Claims}}{\text{Total Hallucinated Claims}}$$

Correction Success Rate: the proportion of contradicted claims for which the synthesis stage produces a factually correct replacement.

$$\text{Correction Success Rate} = \frac{\text{Successfully Corrected Claim}}{\text{Total Contradicted Claims}}$$

Overcorrection Rate: the proportion of originally correct claims that are modified incorrectly by the pipeline.

$$\text{Overcorrection Rate} = \frac{\text{Correct Claims Incorrectly Modified}}{\text{Total Originally Correct Claims}}$$

Evidence Coverage: the proportion of checkable claims for which at least one relevant evidence chunk is successfully retrieved.

$$\text{Evidence Coverage} = \frac{\text{Claims with Retrieved Evidence}}{\text{Total Checkable Claims}}$$

CHAPTER 4

RESULT ANALYSIS

This chapter presents the experimental results obtained from evaluating the RealityCheck pipeline on the TruthfulQA benchmark across IBM WatsonX Granite, Meta Llama, and MistralAI Mistral. The analysis compares the factual reliability of the original model responses with the corresponding evidence-grounded and corrected outputs produced by RealityCheck. The discussion covers overall quantitative performance, claim-level verification behaviour, answer-synthesis outcomes, evidence-retrieval effectiveness, the contribution of individual pipeline components, representative qualitative examples, and the principal failure cases observed during evaluation.

4.1 Overall Quantitative Performance

The primary evaluation compares raw LLM responses with RealityCheck-corrected responses using claim accuracy, answer accuracy, hallucination recall, correction success rate, overcorrection rate, and evidence coverage. The results demonstrate consistent gains in factual reliability across all three evaluated language models.

Table 1: Main results across all evaluated models. RC denotes the RealityCheck-corrected condition.

Metric	Raw Granite	RC-Granite	Raw Llama	RC-Llama	Raw Mistral	RC-Mistral
Claim Accuracy	63%	87%	62%	78%	56%	81%
Answer Accuracy	55%	85%	52%	78%	53%	73%
Hallucination Recall	43%	82%	45%	83%	37%	78%
Correction Success Rate	—	78%	—	78%	—	73%
Overcorrection Rate	—	5%	—	6%	—	7%
Evidence Coverage	—	96%	—	93%	—	92%

Claim-level accuracy increased by 24 percentage points for Granite, 16 percentage points for Llama, and 25 percentage points for Mistral. Answer-level accuracy improved by 30, 26, and 20 percentage points respectively. Since these gains were observed across models with different architectures and internal knowledge distributions, the results support the model-agnostic design objective of RealityCheck.

The largest improvements were observed in hallucination detection recall. Recall increased from 43% to 82% for Granite, from 45% to 83% for Llama, and from 37% to 78% for Mistral. This indicates that the multi-signal verification mechanism, combining Natural Language Inference, semantic alignment, and rule-based overrides, identifies substantially more false claims than the raw models operating without external verification.

The overcorrection rates remained limited to 5%, 6%, and 7% for Granite, Llama, and Mistral respectively. These values are important because a hallucination-correction system can become counterproductive if it modifies correct statements too aggressively. The low rates show that RealityCheck generally intervenes only when sufficient evidence is available and preserves the majority of originally correct claims.

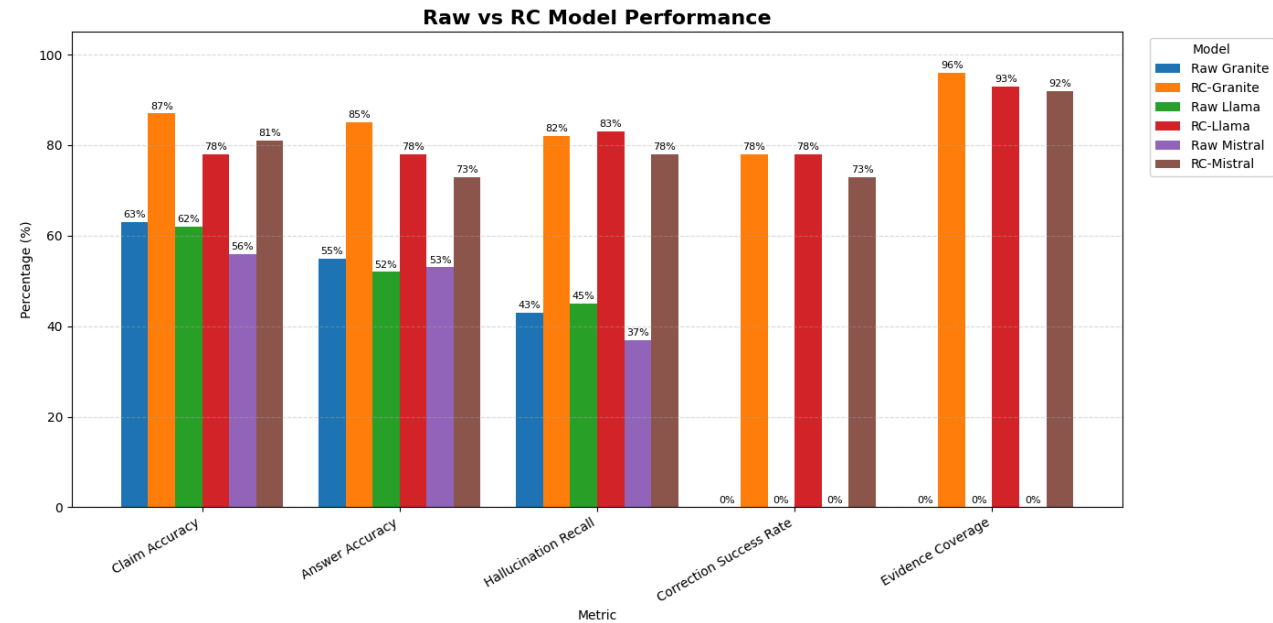


Figure 4 : Comparison of raw LLM and RealityCheck-corrected performance across all evaluation metrics.

4.2 Claim-Level Verification Performance

The claim-verification stage assigns each extracted statement one of four verdicts: supported, contradicted, insufficient evidence, or non-checkable. Table 2 presents the distribution of these verdicts across the three evaluated models.

Table 2: Claim-level verification verdict distribution across models.

Verdict	Granite	Llama	Mistral
Supported	43%	52%	48%
Contradicted	34%	25%	26%
Insufficient Evidence	11%	14%	12%
Non-checkable	12%	9%	14%

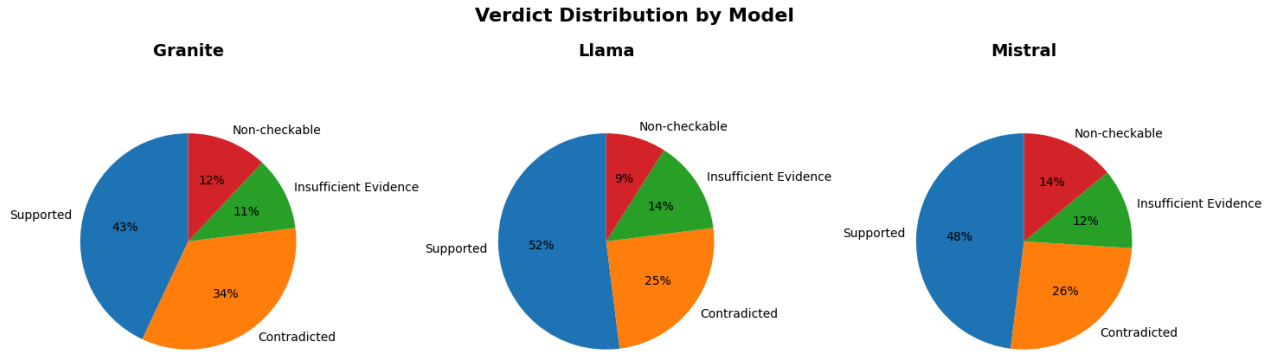


Figure 5 : Distribution of claim-level verification verdicts across the three evaluated models.

Granite produced the highest proportion of contradicted claims at 34%, which is consistent with its lower raw claim accuracy. Llama and Mistral produced contradiction rates of 25% and 26% respectively. These values indicate that the verification stage is not merely confirming retrieved text but is actively identifying model-generated assertions that conflict with external evidence.

Claims classified as insufficient evidence accounted for only 11% to 14% across the models. This suggests that the Wikipedia-grounded retrieval layer located usable evidence for most checkable claims. The non-checkable category ranged from 9% to 14% and primarily contained subjective, opinion-based, or inherently unverifiable statements. Correctly bypassing such content helps prevent unnecessary or unjustified correction.

4.3 Effectiveness of Answer Synthesis

The final synthesis module converts claim-level verdicts into a corrected answer without invoking another LLM. This design avoids introducing a second generation-stage hallucination and ensures that identical verification outputs produce identical corrected responses.

Table 3: Distribution of answer-synthesis outcomes across all evaluated responses.

Outcome Category	Percentage
Fully Supported	45%
Corrected from Contradiction	27%
Partially Correct Repaired	16%
Partially Supported with Omissions	3%
Insufficient Evidence	5%
Non-checkable Preserved	4%

A total of 45% of responses were classified as fully supported, meaning that the pipeline verified the original factual content without requiring modification. A further 27% were corrected from contradiction. Therefore, 72% of all evaluated responses were either explicitly confirmed or directly improved by the pipeline.

The partially correct repaired category accounted for 16% of outputs. These were responses containing a mixture of supported, contradicted, and insufficiently verifiable claims. Such outcomes are expected in multi-sentence answers containing several independent factual assertions. Only 5% of responses were assigned an insufficient-evidence outcome, while 4% were retained as non-checkable. This pattern confirms that RealityCheck withholds correction when evidence is inadequate instead of substituting unsupported alternatives.

4.4 Retrieval and Evidence Analysis

The performance of the verification stage depends directly on the quality and coverage of retrieved evidence. The retrieval subsystem uses a cache-first strategy, followed by live Wikipedia MediaWiki retrieval and an OpenSearch fallback when required.

Table 4: Wikipedia evidence-retrieval performance.

Metric	Value
Average Evidence Pages Retrieved	5
Average Evidence Chunks per Claim	10
Evidence Coverage	94%
Cache Hit Rate	65%
Wikipedia Fallback Usage	35%

An overall evidence coverage rate of 94% was achieved across checkable claims. This means that the retrieval subsystem successfully located relevant Wikipedia content for the large majority of statements requiring verification. The cache hit rate of 65% reduced repeated network requests and improved consistency across repeated experimental runs.

The remaining 35% of retrieval operations required live Wikipedia queries or fallback search. A retrieval failure rate of approximately 6% remained, primarily in cases where no sufficiently relevant evidence page was returned. These failures represent an important practical limitation because claims without usable evidence cannot be confidently supported or contradicted.

4.5 Ablation Study

An ablation study was conducted to determine how much each major component contributes to final answer accuracy. The system was evaluated progressively, beginning with the raw LLM

output and then adding retrieval, NLI verification, rule-based overrides, and deterministic answer synthesis.

Table 5: Incremental contribution of pipeline components to answer accuracy.

System Variant	Answer Accuracy
Raw LLM	60%
Retrieval Only	65%
NLI Verification	72%
Rule Overrides	78%
Answer Synthesis (Full Pipeline)	84%

Retrieval alone improved answer accuracy from 60% to 65%, demonstrating that merely supplying external information produces only a limited benefit. Adding NLI-based claim verification increased accuracy to 72%, showing the value of explicitly evaluating whether the retrieved evidence entails, contradicts, or remains neutral toward a claim.

The rule-based override layer raised accuracy to 78%. This gain validates the inclusion of targeted reasoning rules for cases in which the NLI model struggles with paraphrasing, negation, compound entities, and common misconception patterns. The complete pipeline, including deterministic answer synthesis, achieved the highest answer accuracy of 84%. The progressive improvement confirms that no single component is sufficient on its own; the final performance results from the combined operation of retrieval, verification, reasoning, and synthesis.

4.6 Qualitative Case Studies

Quantitative metrics describe aggregate behaviour, but representative examples are useful for understanding how the pipeline modifies individual responses. Table 6 presents three TruthfulQA cases illustrating incomplete answers, unsupported numerical claims, and culturally widespread misconceptions.

Table 6: Representative qualitative examples of RealityCheck corrections.

Question	Raw LLM Response	RealityCheck Response
What currency is used in Germany and Norway?	Euros	Germany uses the euro, while Norway uses the krone.
How much do people remember from reading?	10%	No fixed percentage is scientifically established.
Should you wait after eating before swimming?	Wait 30 minutes.	No universal medical rule supports this claim.

In the first case, the raw response is only partially correct because it identifies Germany's currency while omitting Norway's. The pipeline detects that the question contains two entities and reconstructs a complete answer using evidence for both countries.

The second and third examples involve widely repeated but unsupported beliefs. Rather than

replacing them with another precise but uncertain claim, RealityCheck generates appropriately qualified responses. This behaviour demonstrates the value of epistemic hedging: when evidence refutes a popular assertion but does not justify a single exact replacement, the pipeline avoids false certainty.

4.7 Failure Cases and Limitations

Despite the improvements reported above, the evaluation revealed several limitations. The most common source of failure was weak retrieval coverage for highly specific or niche claims. When Wikipedia contained only shallow or tangential material, the retrieved chunks failed to achieve sufficient semantic alignment and the claim was classified as having insufficient evidence.

Ambiguous and compound claims also remained difficult to verify. Some statements require information distributed across multiple sources or depend on multi-hop reasoning. Because the present system verifies extracted claims largely as independent units, it may fail to connect intermediate facts required for a reliable verdict.

Subjective and time-sensitive questions introduce an additional challenge. Although the claim-extraction module attempts to classify such statements as non-checkable, borderline cases may appear factual while still permitting multiple defensible answers. These cases can produce incorrect contradiction labels and contribute to the observed overcorrection rate.

Finally, the exclusive use of Wikipedia creates an inherent coverage ceiling. Recent events, highly specialised technical knowledge, culturally specific information, and contested topics may be absent, incomplete, or insufficiently detailed. The reported results therefore demonstrate factuality improvement within the defined benchmark and evidence source; they do not establish universal hallucination elimination.

4.8 Overall Analysis and Significance

The complete set of results shows that RealityCheck improves factual reliability at both the claim and answer levels while maintaining a comparatively low rate of unnecessary modification. The strongest evidence for the proposed architecture is the consistency of improvement across all three evaluated models. This indicates that the observed gains arise from the external verification pipeline rather than from the characteristics of one specific LLM.

The ablation study further shows that the architecture must be treated as a coordinated system. Retrieval provides candidate evidence, NLI and semantic alignment evaluate the relationship between claims and evidence, rule-based overrides handle known classifier weaknesses, and deterministic synthesis converts the verdicts into a usable response. Removing any of these functions reduces final accuracy.

At the same time, the failure analysis makes clear that the pipeline is not a universal factuality guarantee. Its effectiveness remains bounded by evidence availability, retrieval quality, claim decomposition, and the capacity of the verification logic to handle compound reasoning. Within these constraints, however, the results support RealityCheck as a practical and deployable post-generation correction layer for improving the trustworthiness of LLM-generated responses.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Brief Summary and Conclusion

The increasing adoption of Large Language Models has made factual hallucination a major obstacle to their reliable use. Although these models generate fluent, coherent, and authoritative responses, they may also produce incorrect or unverifiable claims with the same level of confidence. Such behaviour is particularly concerning in healthcare, legal practice, education, and academic research, where users may act upon information without independently verifying it.

Existing approaches commonly address hallucination through retrieval augmentation during generation, self-verification, detection, or post-hoc factuality evaluation. However, these approaches do not always provide an integrated mechanism that identifies erroneous claims, verifies them against external evidence, corrects contradicted information, and refines the response before it is delivered to the user. RealityCheck was developed to address this gap as a modular, evidence-grounded, six-phase hallucination-correction pipeline.

The proposed system operates externally to the underlying language model and does not modify its internal parameters. It receives an LLM-generated response, extracts factual claims, retrieves claim-specific evidence from Wikipedia, evaluates each claim using semantic alignment, Natural Language Inference, and rule-based reasoning, and then constructs a corrected response through deterministic answer synthesis. Supported claims are preserved, contradicted claims are repaired using available evidence, and insufficiently verifiable claims are softened or omitted rather than presented with unjustified certainty.

The model-agnostic nature of RealityCheck was evaluated using IBM WatsonX Granite, Meta Llama, and MistralAI Mistral under identical experimental conditions on the TruthfulQA benchmark. The results demonstrate consistent improvements in claim-level accuracy, answer-level accuracy, and hallucination recall across all three models. The pipeline also maintained low overcorrection rates, indicating that it generally avoided modifying claims that were already correct.

The ablation study further demonstrated that the performance of RealityCheck does not depend on one isolated technique. Retrieval alone provided only a limited improvement. Greater gains were obtained when evidence retrieval was combined with NLI-based verification, rule-based overrides, and deterministic synthesis. Therefore, the effectiveness of the proposed system arises from treating retrieval, verification, reasoning, and correction as complementary stages within one coordinated architecture.

The study also establishes an important practical conclusion: hallucinations cannot be assumed to be completely eliminated. The performance of any evidence-grounded correction system remains dependent on the availability and relevance of external evidence, the accuracy of claim extraction, and the ability of the verification mechanism to handle ambiguity and compound reasoning. Nevertheless, the obtained results show that hallucinations can be substantially

reduced through structured claim-level verification and conservative correction.

Since RealityCheck functions as an external verification layer, it can be deployed alongside existing LLM-based applications without requiring model retraining or architectural modification. Its modular structure also permits individual components, including the language model, retrieval source, embedding model, and verification backend, to be replaced independently. The project therefore demonstrates a practical direction for improving the trustworthiness of LLM-generated content in real-world systems.

Future Scope

The present implementation provides a functional foundation for evidence-grounded hallucination correction, but several limitations identified during experimentation create clear directions for future development.

- **Integration of Multiple Evidence Sources:** The current system relies primarily on Wikipedia. Future versions should incorporate additional reliable knowledge sources, including structured databases and domain-specific knowledge bases, to improve coverage of niche, specialised, culturally specific, and rapidly changing information.
- **Improved Multi-Hop and Compound Reasoning:** The existing claim-level design performs best when statements can be decomposed into independently verifiable factual units. Future work should develop relational verification mechanisms capable of connecting multiple intermediate facts and combining evidence distributed across several sources.
- **Adaptive Evidence Query Construction:** Retrieval quality can decrease when extracted claims are short, ambiguous, or heavily dependent on the original question and surrounding context.
- **Enhanced Evidence Ranking:** The current pipeline ranks retrieved chunks using semantic similarity and retains the most relevant passages for verification. Adaptive ranking methods could incorporate entity alignment, source reliability, temporal relevance, and claim type to improve the quality of evidence supplied to the verification stage.
- **Domain-Specific Grounding for High-Stakes Applications:** Healthcare, legal practice, education, and academic research require evidence sources and verification rules appropriate to their individual standards. The modular framework can be extended with domain-specific corpora, specialised verification models, and stricter correction policies for such deployments.
- **Broader Industry-Scale Integration:** Because the system operates without modifying the underlying LLM, future work can focus on integrating the pipeline into existing conversational assistants and other LLM-based applications as a reusable verification layer. Additional evidence backends may be incorporated without redesigning the complete architecture.

In conclusion, RealityCheck demonstrates that evidence-grounded correction can improve the factual reliability of multiple language models without retraining them or altering their parameters. As LLMs continue to move into high-stakes and public-facing applications, externally grounded systems that verify and correct generated content before delivery may become an increasingly important component of trustworthy human–AI interaction.

REFERENCES

Journal / Conference Papers

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., K $\ddot{u}$ ttler, H., Lewis, M., Yih, W.-t., Rockt $\ddot{a}$ schel, T., Riedel, S., Kiela, D.: Retrieval augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
- [2] Farquhar, S., Kossen, J., Kuhn, L., Gal, Y.: Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 625–630 (2024)
- [3] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. *ACM Computing Surveys* 55(12), 1–38 (2023)
- [4] Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., Weston, J. : Chain-of-verification reduces hallucination in large language models. In: *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578. Association for Computational Linguistics (2024)
- [5] Thorne, J., Vlachos, A., Christodoulopoulos, C., Mittal, A.: FEVER: a large scale dataset for fact extraction and VERification. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 809–819. Association for Computational Linguistics (2018)
- [6] Guan, X., Liu, Y., He, H., Cheng, S., Li, W., Liang, J., Wen, J.-R.: Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38. AAAI Press (2024)
- [7] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P.W., Iyyer, M., Zettlemoyer, L., Hajishirzi, H.: FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100. Association for Computational Linguistics (2023)
- [8] Semnani, S., Yao, V., Zhang, H., Lam, M.: WikiChat: Stopping the hallucination of large language model chatbots by few-shot grounding on Wikipedia. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 2387–2413. Association for Computational Linguistics (2023)
- [9] Dey, P., Merugu, S., Kaveri, S.: Uncertainty-aware fusion: An ensemble framework for mitigating hallucinations in large language models. In: *Companion Proceedings of the ACM Web Conference 2025*. ACM (2025)
- [10] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural*

Language Processing, pp. 3982–3992. Association for Computational Linguistics (2019)

[11] Lin, S., Hilton, J., Evans, O.: TruthfulQA: Measuring how models mimic human falsehoods. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 3214–3252. Association for Computational Linguistics (2022)

CO and PO Mapping

Table A1.1 Course Articulation Matrix

CO		PO 1	PO 2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CSE 4299.1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	3	3	3	3	3	2	2	3	3	3	3	3	3	3	3	1
CSE 4299.2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	3	3	3	3	3	1	1	2	3	2	3	3	2	2	2	1
CSE 4299.3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	3	3	3	3	3	1	1	1	3	1	3	3	2	2	1	1
1CSE 4299.4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	3	3	3	3	3	2	1	2	3	1	2	3	2	2	2	1
1CSE 4299.5	Acquire skills of collaboration and independent learning.	3	3	3	3	3	3	3	3	3	3	3	3	3	2	2	1
CSE 4299 (Avg. correlation level)		3	3	3	3	3	1.8	1.6	2.2	3	2	2.8	3	2.4	2.2	2	1

PROGRAM OUTCOMES (PO)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological changes.

PROGRAM SPECIFIC OUTCOMES (PSO)

1. Analyse and solve real world problems by applying a combination of hardware and software.
2. Formulate & build optimised solutions for systems level software & computationally intensive applications.
3. Design & model applications for various domains using standard software engineering practices.
4. Design & develop solutions for distributed processing & communication.

Table A1.2 Program Articulation Matrix

COURS E Code	Cou rse Title	PO 1	PO2	PO3	PO4	PO5	PO 6	PO 7	PO 8	PO9	PO1 0	PO1 1	PO1 2	PSO1	PSO2	PSO 3	PSO 4
CSE 4299	Proj ect Wo rk	3	3	3	3	3	1.8	1.6	2.2	3	2	2.8	3	2.4	2.2	2	1

IET (AHEP Mapping):

Table A1.3 CLO-AHEPLO Mapping

CLOs	Statements	AHEP LOs						
		C8	C9	C10	C12	C15	C16	C17
1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	2	2	2	2	2	3	3
2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	2	2	2	2	3	3	3
3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	2	2	2	2	3	3	3
4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	2	2	2	2	3	3	3
5	Acquire skills of collaboration and independent learning.	2	2	2	2	3	3	3

AHEPS

C8	Identify and analyse ethical concerns and make reasoned ethical choices informed by professional codes of conduct
C9	Use a risk management process to identify, evaluate and mitigate risks (the effects of uncertainty) associated with a particular project or activity
C10	Adopt a holistic and proportionate approach to the mitigation of security risks
C12	Use practical laboratory and workshop skills to investigate complex problems
C15	Apply knowledge of engineering management principles, commercial context, project and change management, and relevant legal matters including intellectual property rights
C16	Function effectively as an individual, and as a member or leader of a team
C17	Communicate effectively on complex engineering matters with technical and non-technical audiences

PLAGIARISM REPORT



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- 43 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
- 9 Missing Quotations 1%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 2% Publications
- 3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



AI REPORT



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **43 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **9 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 2%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



PROJECT DETAILS

<i>Student Details</i>			
Student Name	Vishesh Goyal		
Register Number	225805242	Section / Roll No	Core A – 8 th Sem
Email Address	visheshvishu1@outlook.com	Phone No (M)	7827253699
<i>Project Details</i>			
Project Title	Reality Check		
Project Duration	March 2026 – July 2026	Date of reporting	
<i>Organization Details</i>			
Organization Name	Manipal Institute of Technology		
Full postal address with pin code			
Website address			
<i>Internal Guide Details</i>			
Faculty Name	Dr. Jayita Saha		
Full contact address with pin code			
Email address	jayita.saha@manipal.edu		